Time-Series Data

append, compress, roll up, expire

Watch the 4-min video

In one line: time-series data is append-only points of (timestamp, value, tags) queried by time range and aggregated; store it in a time-series database that writes **time-chunked, heavily compressed** data, batch the writes, **downsample** into rollups with **retention per resolution**, and keep tag **cardinality** low.

1 A million points a second, forever

A factory has 200,000 sensors reporting temperature, vibration and pressure every second: around 600,000 values a second, with headroom planned for 1 million. At 1 million a second that is 86.4 billion points a day. Engineers want live charts, alerts within seconds, and trends going back years. A normal relational table with a B-tree index per row would spend its life updating indexes and still hold terabytes a day. The data's shape allows a much better design.

2 The shape of time-series data

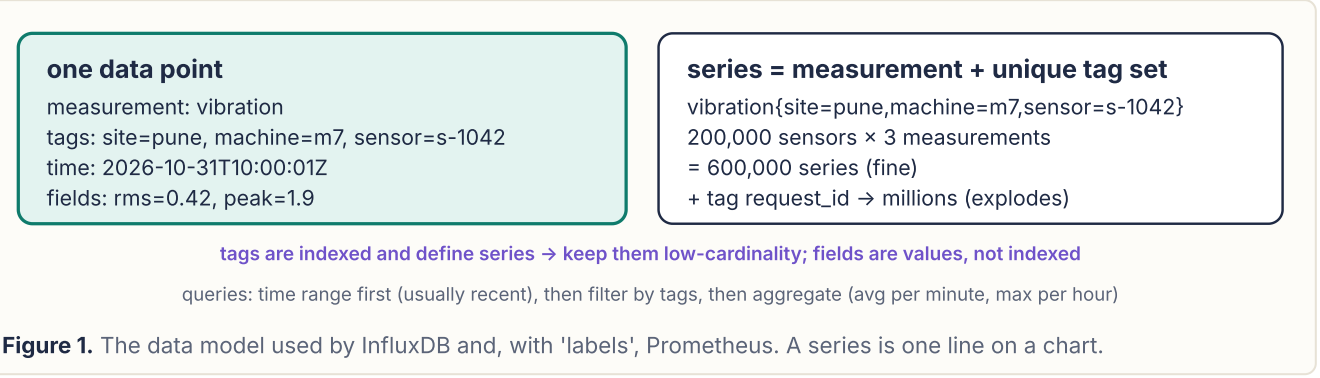


Figure 1. The data model used by InfluxDB and, with 'labels', Prometheus. A series is one line on a chart.

- **Writes** are constant, append-only and almost always for "now"; updates and deletes of single points are rare.
- **Reads** ask for a time range, usually recent, filtered by tags, and aggregated: average per minute, 99th percentile per hour.
- **Value decays** with age: per-second detail matters for days, hourly trends for years.

3 How time-series databases store it

Recent points sit in an in-memory **head** (with a write-ahead log for durability), then are flushed into immutable **time chunks** stored column by column: TimescaleDB calls them chunks of a hypertable, Prometheus uses 2-hour blocks compacted into larger ones, InfluxDB uses shards by time. A query for the last hour opens only the chunks that overlap it.

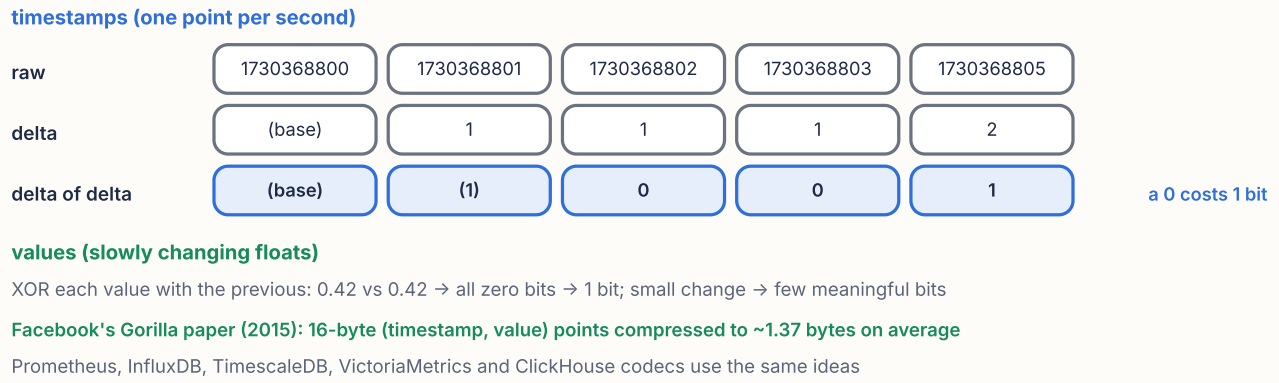


Figure 2. Why compression is so good: regular timestamps reduce to runs of zeros, and slowly changing values XOR to mostly zero bits.

With ~1.4 bytes a point, a day of 86.4 billion points is about **120 GB** instead of ~1.4 TB of raw 16-byte points. Columnar chunks also make aggregates fast, because a query reads one tightly packed column.

System	Model	Good for
Prometheus	pull-based scraping, local TSDB, PromQL	infrastructure metrics and alerting; pair with Thanos, Mimir or VictoriaMetrics for long-term storage
InfluxDB	push writes (line protocol), tags + fields	IoT and app metrics
TimescaleDB	PostgreSQL extension: hypertables, continuous aggregates	time series that need SQL and joins with business tables
ClickHouse	columnar OLAP with MergeTree tables	very high volume events and logs, ad-hoc analytics

4 Rollups and retention

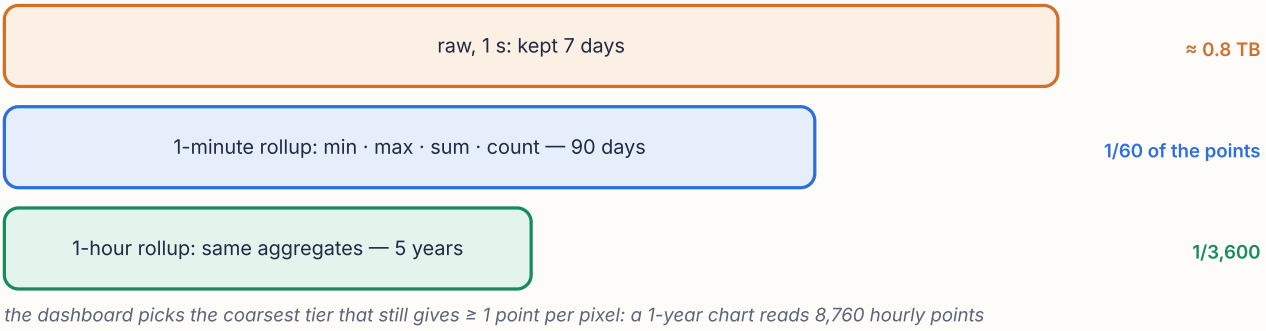


Figure 3. Three resolutions, each with its own retention. Storage is dominated by the short-lived raw tier.

A **rollup** (downsample) aggregates raw points into coarser buckets, continuously: TimescaleDB continuous aggregates, Prometheus recording rules, InfluxDB tasks. Store **min, max, sum and count**, not just the average: averages of averages are wrong when buckets have different counts, and max is what alerts and capacity planning need. (Percentiles need sketches, such as t-digest, because they can't be merged from per-minute percentiles.)

Retention runs per tier: raw after 7 days, minutes after 90, hours after 5 years. Because data lives in time chunks, expiry **drops a whole chunk** (a file delete), which is instant, unlike `DELETE FROM … WHERE ts < …` over billions of rows.

5 Cardinality: the hidden danger

Each unique tag combination is a series, and a TSDB keeps an index entry and per-series buffers for every active one. 200,000 sensors × 3 measurements is 600,000 series, fine. Add a tag with unbounded values (request ID, user ID, full URL, error message) and you create millions of series that each hold a handful of points: the index explodes, memory runs out, compression collapses. Rules: tags only for values you group or filter by with a bounded set; unique values go in fields or in logs and traces; monitor the active-series count and limit it per tenant.

WORKED EXAMPLE: IOT SENSORS AT A MILLION POINTS PER SECOND

Sensors publish over **MQTT** (a lightweight pub/sub protocol for devices) to regional gateways, which forward into **Kafka** partitioned by sensor ID, so a database outage never loses data. Ingest workers consume Kafka and write **batches of 5,000–10,000 points** per request into the TSDB, which is partitioned by time and by sensor. Raw data lives 7 days (~0.8 TB compressed); 1-minute rollups 90 days; hourly rollups 5 years. Alerts ("vibration rms > 1.5 for 30 s") run on the Kafka stream in a stream processor (lesson 32), not by polling the database. A dashboard zoomed to the last hour reads raw points; one showing the last year reads 8,760 hourly points per series instead of 31 million raw ones.

TRADE-OFFS

- Longer raw retention allows detailed forensics but multiplies the largest tier of storage.
- Bigger write batches raise throughput but add latency and data lost in a crash (unless buffered in Kafka).
- Downsampling loses detail: a 2-second spike disappears from an hourly average; keep max to preserve it.
- A purpose-built TSDB is fast and compact; Postgres + Timescale gives SQL and joins with somewhat less efficiency at extreme scale.

COMMON MISTAKES

- Putting user IDs, request IDs or URLs into tags.
- Writing one point per request instead of batching thousands.
- Storing only averages in rollups, so peaks and correct re-aggregation are lost.
- Deleting old data row by row instead of dropping time chunks.

6 Key terms

Series: one measurement with one unique tag set

Time chunk: time-bounded partition of the data, dropped whole

Rollup: pre-aggregated coarser copy (min, max, sum, count)

Cardinality: number of distinct series

Delta-of-delta / XOR: compression for timestamps / floats

Retention policy: how long each resolution is kept

7 Interview questions

Q1. How do you store and query high-volume time-series data? BASIC

A TSDB with append-only, time-chunked, compressed storage; batched writes; rollups plus retention per resolution; queries read the coarsest table that answers them.

Q2. What's the main pitfall? BASIC

High cardinality: tags with too many unique values create millions of series and blow up the index and memory. Keep tags low-cardinality.

Q3. Estimate storage for 1M points/s kept raw for 7 days. SCENARIO

86.4 billion points a day × ~1.4 bytes ≈ 120 GB/day, so ~0.85 TB for 7 days, versus ~10 TB at 16 bytes a point uncompressed.

Q4. A year-long chart takes 40 seconds to load. Fix it. SCENARIO

Serve it from an hourly (or daily) rollup chosen by zoom level, so it reads thousands of points, not billions.

Q5. Why store min, max, sum and count in rollups instead of the average? TRICKY

Sum and count re-aggregate correctly into coarser buckets and give the average; max and min keep spikes an average hides.

Q6. Why is deleting old time-series data cheap in a TSDB but expensive in a regular table? TRICKY

Data is partitioned into time chunks, so retention drops whole files; a regular table must find and delete billions of rows and update indexes.

Next: SD-Q-011 · Geospatial Indexing backendroadmap.com